

DOCUMENTACIÓN TÉCNICA

Sistema Marketplace

API REST de e-commerce — arquitectura, seguridad, flujos de negocio y referencia completa de endpoints

Java

Spring Boot 4.1.0

Spring Data JPA

Hibernate

MySQL 8

Spring Security

JWT (jjwt 0.12.5)

Lombok

Maven

Generado a partir del código fuente del proyecto (paquete `com.uade.tpo.marketplace`) · Septiembre 2026

1. Arquitectura general del sistema
2. Conceptos fundamentales: REST, JPA, Hibernate y DTOs
3. Anotaciones utilizadas en el proyecto
4. Modelo de datos (entidades y relaciones)
5. Seguridad: autenticación JWT y autorización por rol
6. Manejo de errores y excepciones
7. Referencia de controllers y endpoints (los 10 controllers)
8. Flujos de negocio end-to-end
9. Configuración del proyecto (application.properties, pom.xml)

1. Arquitectura general del sistema

El proyecto es un backend de e-commerce construido como **API REST con Spring Boot**, organizado en capas siguiendo el patrón clásico **Controller** → **Service** → **Repository** → **Entity**, con una capa adicional de **DTOs** (Data Transfer Objects) para no exponer las entidades de persistencia directamente al cliente.

Cliente (Postman / frontend web / mobile) — envía peticiones HTTP con JSON y, salvo excepciones públicas, un header `Authorization: Bearer <JWT>`.



Filtro de seguridad (`JwtAuthenticationFilter`) — intercepta cada request, valida el JWT y carga el usuario autenticado en el contexto de seguridad de Spring.



Controllers (`@RestController`) — reciben la petición HTTP, la traducen a un DTO *Request*, delegan en el service correspondiente y devuelven un DTO *Response* envuelto en `ResponseEntity`.



Services (interfaz `I...Service` + implementación `...ServiceImpl`) — contienen la lógica de negocio: validaciones, reglas de stock, cálculo de descuentos, orquestación de checkout, etc. Marcados con `@Transactional`.



Repositories (interfaces que extienden `JpaRepository`) — acceso a datos. Spring Data JPA genera la implementación automáticamente en tiempo de ejecución.



Entities (`@Entity`) — clases Java mapeadas 1 a 1 con tablas de la base de datos MySQL, gestionadas por Hibernate.

Módulos del dominio

El sistema modela un flujo de e-commerce completo con 9 entidades principales:

Módulo	Responsabilidad
Usuario	Cuentas, autenticación, roles (USER / ADMIN)
Category / Producto / Imagen	Catálogo de productos
Descuento	Promociones por producto con vigencia por fecha
Carrito / DetalleCarrito	Carrito de compras de cada usuario
Pedido / DetallePedido	Órdenes de compra confirmadas
Auth (soporte)	Login/registro y emisión de JWT

Por qué interfaz + implementación: cada servicio se define como una interfaz (`ICarritoService`) y una clase que la implementa (`CarritoServiceImpl`). Los controllers dependen de la interfaz, no de la clase concreta — esto es *inversión de dependencias*: permite cambiar la implementación (o mockearla en tests) sin tocar el controller. Spring inyecta automáticamente la implementación con `@Autowired`.

2. Conceptos fundamentales

¿Qué es una API REST?

REST (Representational State Transfer) es un estilo de arquitectura para diseñar servicios web. Una API REST expone **recursos** (en este proyecto: productos, carritos, pedidos, usuarios...) identificados por URLs, y se interactúa con ellos usando los **verbos HTTP** estándar:

Verbo	Uso típico en este proyecto
GET	Leer un recurso o una lista (no modifica nada, es idempotente)
POST	Crear un recurso nuevo, o ejecutar una acción (ej. <code>/checkout</code> , <code>/cancelar</code>)
PUT	Reemplazar/actualizar por completo un recurso existente
PATCH	Actualizar parcialmente un recurso (ej. solo el stock, solo el rol)
DELETE	Eliminar un recurso

Cada respuesta usa un **código de estado HTTP** (200 OK, 201/200 creado, 204 sin contenido, 404 no encontrado, 400 solicitud inválida, 401 no autenticado, 403 sin permisos) para que el cliente sepa qué pasó sin tener que parsear el cuerpo de la respuesta. La API es **stateless**: el servidor no guarda sesión entre requests, cada petición debe traer su propio JWT.

¿Qué es Spring Boot?

Es un framework de Java que simplifica la creación de aplicaciones Spring: trae un servidor HTTP embebido (Tomcat), auto-configuración basada en las dependencias del `pom.xml`, e inyección de dependencias mediante anotaciones. En este proyecto arranca en el puerto 4002 (ver `application.properties`).

¿Qué es JPA?

JPA (Java Persistence API) es una **especificación** (un conjunto de interfaces y reglas, no una librería concreta) que define cómo mapear objetos Java a tablas de una base de datos relacional — esto se llama **ORM** (Object-Relational Mapping). JPA define anotaciones como `@Entity`, `@Id`, `@ManyToOne`, el lenguaje de consultas **JPQL** (parecido a SQL pero sobre objetos Java en vez de tablas), y el concepto de `EntityManager` como puerta de entrada a la persistencia.

¿Qué es Hibernate?

Es la **implementación** más usada de JPA (JPA es la interfaz, Hibernate es quien realmente hace el trabajo). Es el motor que:

- Traduce las entidades `@Entity` a sentencias SQL reales para MySQL.
- Gestiona el **Persistence Context**: un caché de primer nivel que trackea los objetos cargados en una transacción para detectar cambios (*dirty checking*) y generar los UPDATE automáticamente al hacer commit, sin necesidad de llamar a `.save()` explícitamente en muchos casos.
- Con `spring.jpa.hibernate.ddl-auto=update` (configurado en este proyecto), genera y actualiza automáticamente el esquema de tablas de MySQL a partir de las entidades cada vez que arranca la aplicación.

Nota sobre `ddl-auto=update`: es muy cómodo en desarrollo (no hay que escribir migraciones a mano), pero es riesgoso en producción porque Hibernate puede alterar el esquema de forma inesperada. En un entorno productivo se recomienda `validate` + una herramienta de migraciones versionadas (Flyway/Liquibase).

Spring Data JPA y JpaRepository

Spring Data JPA es una capa sobre JPA/Hibernate que elimina el código repetitivo de acceso a datos. Cada repositorio del proyecto (ej. `IProductoRepository`) es solo una **interfaz** que extiende `JpaRepository<Entidad, TipoId>`: Spring genera la implementación en tiempo de ejecución, con métodos gratis como `findAll()`, `findById()`, `save()`, `deleteById()`, y paginación (`Page<T>`, `PageRequest`).

Además se pueden declarar **métodos derivados por nombre** (Spring interpreta el nombre del método y genera la consulta, ej. `findByUsername(String username)`) o consultas explícitas con `@Query` en JPQL, como esta usada para el buscador de productos con filtros opcionales:

```
@Query("select p from Producto p where "
      + "(:categoriaId is null or p.categoria.id = :categoriaId) and "
      + "(:nombre is null or lower(p.nombreProducto) like lower(concat('%', :nombre, '%')))) and "
      + "(:precioMin is null or p.precioUnitario >= :precioMin) and "
      + "(:precioMax is null or p.precioUnitario <= :precioMax)")
Page<Producto> buscarProductos(Integer categoriaId, String nombre,
                              Double precioMin, Double precioMax, Pageable pageable);
```

Nota que la consulta opera sobre **entidades y campos Java** (Producto, p.categoria.id), no sobre nombres de tablas/columnas SQL — esa traducción la hace Hibernate.

El patrón DTO (Request / Response)

Las entidades JPA (Usuario, Producto...) nunca se devuelven directamente al cliente. En su lugar, cada recurso tiene:

- **DTO Request** (entity/dto/request/): representa el JSON que el cliente envía en el body (ej. ProductoRequest).
- **DTO Response** (entity/dto/response/): representa el JSON que se devuelve, con un método estático from(entidad) que arma la respuesta a partir de la entidad (ej. ProductoResponse.from(producto)).

Por qué: evita exponer campos sensibles (como contraseñas de Usuario), evita ciclos infinitos de serialización JSON en relaciones bidireccionales, y desacopla el contrato público de la API de los detalles internos del modelo de datos.

3. Anotaciones utilizadas en el proyecto

Persistencia (JPA / Hibernate)

Anotación	Uso en el proyecto
<code>@Entity</code>	Marca la clase como mapeada a una tabla de MySQL (ej. <code>Usuario</code> , <code>Producto</code>).
<code>@Id</code>	Marca el campo como clave primaria de la tabla.
<code>@GeneratedValue(strategy = GenerationType.IDENTITY)</code>	El valor del ID lo autogenera MySQL (auto-incremental), no la aplicación.
<code>@Column</code>	Marca un atributo como columna de la tabla (opcionalmente con nombre/longitud distinta).
<code>@ManyToOne + @JoinColumn</code>	Relación "muchos a uno" con clave foránea, ej. muchos <code>Producto</code> pertenecen a una <code>Category</code> ; muchos <code>DetalleCarrito</code> pertenecen a un <code>Carrito</code> .
<code>@OneToMany(mappedBy=...)</code>	Lado inverso de la relación (ej. un <code>Usuario</code> tiene muchos <code>Pedido</code>).
<code>@Enumerated(EnumType.STRING)</code>	Persiste un enum de Java (<code>Role.USER</code> / <code>Role.ADMIN</code>) como texto legible en la columna, en vez de un número ordinal.
<code>@Lob</code>	Marca un campo como objeto grande binario (BLOB) — usado en <code>Imagen.datos</code> para guardar el archivo de imagen subido.

Spring Web (Controllers)

Anotación	Uso en el proyecto
<code>@RestController</code>	Marca la clase como controller REST: cada método devuelve directamente el body de la respuesta (JSON), no una vista.
<code>@RequestMapping("Producto")</code>	Define el prefijo de ruta común de todos los endpoints de la clase (ej. todo <code>ProductoController</code> cuelga de <code>/Producto</code>).
<code>@GetMapping</code> / <code>@PostMapping</code> / <code>@PutMapping</code> / <code>@PatchMapping</code> / <code>@DeleteMapping</code>	Asocian un método Java a un verbo HTTP + sub-ruta.
<code>@PathVariable</code>	Extrae un valor de la URL, ej. el <code>{productoId}</code> de <code>/Producto/{productoId}</code> .
<code>@RequestParam(required = false)</code>	Extrae un query param opcional (ej. <code>?nombre=...&page=0&size=10</code>) usado para filtros y paginación.
<code>@RequestBody</code>	Deserializa el JSON del body de la petición al DTO Java correspondiente.

Inyección de dependencias y capas

Anotación	Uso en el proyecto
<code>@Service</code>	Marca una clase como componente de la capa de lógica de negocio (ej. <code>ProductoServiceImpl</code>).
<code>@Repository</code>	Marca una interfaz como componente de acceso a datos (los <code>I...Repository</code>).
<code>@Component</code>	Componente genérico gestionado por Spring (ej. filtros y handlers de seguridad).
<code>@Configuration</code> / <code>@Bean</code>	Clase de configuración que declara manualmente objetos (<i>beans</i>) que Spring debe gestionar, ej. <code>PasswordEncoder</code> , <code>AuthenticationManager</code> en <code>ApplicationConfig</code> .
<code>@Autowired</code>	Le pide a Spring que inyecte automáticamente una dependencia (normalmente la implementación de una interfaz) en el campo.
<code>@RequiredArgsConstructor</code> (Lombok)	Genera un constructor con los campos <code>final</code> , usado como alternativa a <code>@Autowired</code> para inyección por constructor (ej. <code>SecurityConfig</code> , <code>JwtAuthenticationFilter</code>).

<code>@Transactional(rollbackFor = Throwable.class)</code>	Envuelve el método en una transacción de base de datos: si algo falla, se revierten todos los cambios (fundamental en <code>checkout()</code> , que toca 4 tablas).
<code>@Transactional(readOnly = true)</code>	Optimización para operaciones de solo lectura (Hibernate evita el chequeo de cambios).
<code>@Value("\${...}")</code>	Inyecta un valor desde <code>application.properties</code> (ej. la clave secreta y expiración del JWT en <code>JwtService</code>).
<code>@ResponseStatus(code=..., reason=...)</code>	Sobre una excepción custom: le dice a Spring qué código HTTP devolver automáticamente si esa excepción no es atrapada (ver sección 6).

Lombok (reducción de código repetitivo)

Anotación	Uso en el proyecto
<code>@Data</code>	Genera getters, setters, <code>equals()</code> , <code>hashCode()</code> y <code>toString()</code> automáticamente. Usada en todas las entidades y DTOs.
<code>@NoArgsConstructor</code> / <code>@AllArgsConstructor</code>	Generan el constructor vacío (requerido por JPA/Hibernate para instanciar entidades por reflexión) y el constructor con todos los campos.
<code>@Builder</code>	Genera un patrón <i>builder</i> fluido, usado en <code>AuthenticationResponse.builder().accessToken(...).build()</code> .

5. Seguridad: JWT y autorización por rol

Flujo de autenticación

- 1 **Registro** — POST /Auth/register con datos del usuario. El service crea el Usuario con `role = USER` y guarda la contraseña **hasheada con BCrypt** (PasswordEncoder), nunca en texto plano.
- 2 **Login** — POST /Auth/authenticate con username/contraseña. El AuthenticationManager de Spring Security valida las credenciales contra la base (comparando el hash).
- 3 **Emisión del token** — si es válido, `JwtService.generateToken()` firma un JWT (HMAC-SHA con la clave de `application.properties`) con el username como *subject* y expiración de 24hs (86400000 ms). Se devuelve como `accessToken`.
- 4 **Peticiones autenticadas** — el cliente debe enviar `Authorization: Bearer <token>` en cada request a un endpoint protegido.
- 5 **Validación por filtro** — `JwtAuthenticationFilter` (se ejecuta una vez por request, antes del filtro estándar de Spring) extrae el token, obtiene el username, carga el `UserDetails` desde la base y verifica firma + expiración. Si es válido, carga la autenticación en el `SecurityContextHolder` para el resto del pipeline.
- 6 **Autorización** — `SecurityConfig` decide, según método HTTP y ruta, si el recurso es público, requiere solo estar autenticado, o requiere rol `ROLE_ADMIN`.

Configuración clave (SecurityConfig)

- **CSRF deshabilitado** — no aplica porque la API es stateless y no usa cookies de sesión para autenticar.
- **Sesión STATELESS** — Spring no crea ni usa `HttpSession`; toda la identidad viaja en el JWT de cada request.
- **SecurityAuthenticationEntryPoint** — responde 401 UNAUTHORIZED con JSON cuando falta o es inválido el token.
- **SecurityExceptionHandler** — responde 403 FORBIDDEN con JSON cuando el usuario está autenticado pero no tiene el rol requerido.

Matriz de autorización por endpoint

Nivel	Significado	Aplica a
PÚBLICO	Sin token, cualquiera puede acceder	POST /Auth/register , POST /Auth/authenticate , lectura (GET) de Producto, Categories, Imagen, Descuento
AUTENTICADO	Requiere JWT válido, cualquier rol	Carrito, DetalleCarrito, Pedido, DetallePedido completos; PUT /Usuario/{id} (editar perfil propio); GET /Usuario/{id}
ADMIN	Requiere JWT + ROLE_ADMIN	Alta/baja/edición de Producto, Categories, Imagen, Descuento; GET/POST /Usuario , PATCH .../permisos , DELETE /Usuario/{id}

Importante: la regla `anyRequest().authenticated()` al final cubre todo lo que no matchea una regla explícita — por eso Carrito/Pedido quedan protegidos "por defecto" sin necesitar reglas propias. Notar también que el sistema **no valida a nivel de código** que un usuario solo vea/edite *su propio* carrito o pedido (por `usuarioId`) — la autorización actual es por rol, no por ownership del recurso; cualquier usuario autenticado (rol USER) puede, por ejemplo, consultar `GET /Carrito/{id}` de otro usuario si conoce el ID.

6. Manejo de errores y excepciones

El proyecto no usa un `@ControllerAdvice` global: cada excepción de negocio es una clase propia anotada con `@ResponseStatus`, que le indica a Spring qué código HTTP devolver automáticamente cuando esa excepción se propaga sin capturar desde el controller. El controller la declara con `throws` y Spring se encarga del resto.

Excepción	HTTP	Mensaje
<code>UsuarioDuplicateException</code>	400	El usuario que se intenta agregar está duplicado
<code>UsuarioNotFoundException</code>	404	Usuario no encontrado
<code>ProductoDuplicateException</code>	400	El producto que se intenta agregar está duplicado
<code>ProductoNotFoundException</code>	404	Producto no encontrado
<code>StockInvalidoException</code>	400	El stock no puede ser negativo
<code>StockInsuficienteException</code>	400	La cantidad solicitada supera el stock disponible del producto
<code>CategoryDuplicateException</code>	400	La categoría que se intenta agregar está duplicada
<code>CategoriaNotFoundException</code>	404	Categoría no encontrada
<code>ImagenDuplicateException</code>	400	La imagen que se intenta agregar está duplicada
<code>DescuentoInvalidoException</code>	400	El porcentaje de descuento debe estar entre 0 y 100
<code>CarritoNotFoundException</code>	404	Carrito no encontrado
<code>CarritoVacioException</code>	400	El carrito no tiene productos para confirmar el pedido
<code>DetalleCarritoDuplicateException</code>	400	El detalle de carrito que se intenta agregar está duplicado
<code>PedidoNotFoundException</code>	404	Pedido no encontrado
<code>DetallePedidoDuplicateException</code>	400	El detalle de pedido que se intenta agregar está duplicado

Autenticación/autorización se maneja aparte, fuera de este mecanismo: `SecurityAuthenticationEntryPoint` (401) y `SecurityExceptionHandler` (403) — ver sección 5 — porque esos errores ocurren en el filtro de seguridad, antes de llegar al controller.

7. Referencia de controllers y endpoints

10 controllers, prefijo base `http://localhost:4002`. Todas las rutas están en la raíz (sin prefijo `/api`).

1. AuthenticationController

`controllers/auth/AuthenticationController.java` · `@RequestMapping("Auth")`

POST `/Auth/register` **PÚBLICO**

Body: `UsuarioRequest` (dni, username, email, nombre, apellido, contrasenia)
Acción: crea el `Usuario` (rol `USER`, password hasheada) y devuelve un JWT ya logueado.
Respuesta: `AuthenticationResponse { accessToken }`
Excepciones: `UsuarioDuplicateException` (400) si el email ya existe

POST `/Auth/authenticate` **PÚBLICO**

Body: `AuthenticationRequest` (username, contrasenia)
Acción: valida credenciales y emite un nuevo JWT.
Respuesta: `AuthenticationResponse { accessToken }`

2. UsuarioController

`controllers/UsuarioController.java` · `@RequestMapping("Usuario")`

Endpoint	Seguridad	Body	Detalle
GET <code>/Usuario?page&size</code>	ADMIN	—	Lista paginada (<code>Page<UsuarioResponse></code>); sin <code>page/size</code> trae todo
GET <code>/Usuario/{usuarioId}</code>	AUTH	—	200 con el usuario o 404
POST <code>/Usuario</code>	ADMIN	<code>UsuarioRequest</code>	Alta manual de usuario (fuera del flujo de registro). Lanza <code>UsuarioDuplicateException</code>
PUT <code>/Usuario/{usuarioId}</code>	AUTH	<code>UsuarioRequest</code>	Actualiza datos del perfil (re-hashea la contraseña enviada)
PATCH <code>/Usuario/{usuarioId}/permisos?role=</code>	ADMIN	query param <code>role</code>	Cambia el rol a <code>USER</code> o <code>ADMIN</code>
DELETE <code>/Usuario/{usuarioId}</code>	ADMIN	—	204 si borró, 404 si no existía

3. CategoriesController

`controllers/CategoriesController.java` · `@RequestMapping("Categories")`

Endpoint	Seguridad	Body	Detalle
GET <code>/Categories?page&size</code>	PÚBLICO	—	Lista paginada de categorías
GET <code>/Categories/{categoryId}</code>	PÚBLICO	—	Detalle o 404
POST <code>/Categories</code>	ADMIN	<code>CategoryRequest{nombre}</code>	Crea categoría. <code>CategoryDuplicateException</code> si ya existe
PUT <code>/Categories/{categoryId}</code>	ADMIN	<code>CategoryRequest</code>	Renombra la categoría
DELETE <code>/Categories/{categoryId}</code>	ADMIN	—	204 / 404

4. ProductoController

controllers/ProductoController.java · @RequestMapping("Producto")

Endpoint	Seguridad	Body	Detalle
GET /Producto? categoriaId&nombre&precioMin&precioMax&page&size	PÚBLICO	—	Búsqueda/filtro combinado (JPQL dinámico, ver sección 2) + paginación
GET /Producto/{productoId}	PÚBLICO	—	Detalle o 404
POST /Producto	ADMIN	ProductoRequest	Crea producto; valida categoría existente y nombre no duplicado
PUT /Producto/{productoId}	ADMIN	ProductoRequest	Actualiza todos los campos
PATCH /Producto/{productoId}/stock	ADMIN	StockRequest{stock}	Fija stock absoluto. StockInvalidoException si es negativo
DELETE /Producto/{productoId}	ADMIN	—	204 / 404

5. ImagenController

controllers/ImagenController.java · @RequestMapping("Imagen")

Endpoint	Seguridad	Body	Detalle
GET /Imagen?productoId	PÚBLICO	—	Lista imágenes, opcionalmente filtradas por producto
GET /Imagen/{imagenId}	PÚBLICO	—	Detalle o 404
POST /Imagen	ADMIN	ImagenRequest{productoId, imageUrl}	Asocia una URL de imagen externa a un producto
POST /Imagen/upload (multipart/form-data)	ADMIN	productoId + archivo image	Sube el binario de la imagen (guardado como @Lob en MySQL), máx. 5MB
PUT /Imagen/{imagenId}	ADMIN	ImagenRequest	Actualiza la URL
DELETE /Imagen/{imagenId}	ADMIN	—	204 / 404

6. DescuentoController

controllers/DescuentoController.java · @RequestMapping("Descuento")

Endpoint	Seguridad	Body	Detalle
GET /Descuento?productoId	PÚBLICO	—	Lista descuentos, opcionalmente por producto
GET /Descuento/{descuentoId}	PÚBLICO	—	Detalle o 404
POST /Descuento	ADMIN	DescuentoRequest	Crea descuento; DescuentoInvalidoException si porcentaje fuera de 0-100
PUT /Descuento/{descuentoId}	ADMIN	DescuentoRequest	Actualiza porcentaje/vigencia/estado activo
DELETE /Descuento/{descuentoId}	ADMIN	—	204 / 404

Cómo se aplica: un descuento se considera *vigente* si activo = true y la fecha actual está dentro de [fechaInicio, fechaFin] (ambos límites opcionales). DescuentoService.getPrecioConDescuento() se invoca al agregar un ítem al carrito, no al momento del checkout.

7. CarritoController

controllers/CarritoController.java · @RequestMapping("Carrito")

Endpoint	Seguridad	Body	Detalle
GET /Carrito?usuarioId	AUTH	—	Lista carritos, opcionalmente por usuario
GET /Carrito/{carritoId}	AUTH	—	Detalle o 404
POST /Carrito	AUTH	CarritoRequest{usuarioId, fechaCarrito}	Crea un carrito nuevo asociado a un usuario
PUT /Carrito/{carritoId}	AUTH	CarritoRequest	Reasigna usuario/fecha
DELETE /Carrito/{carritoId}	AUTH	—	204 / 404
POST /Carrito/{carritoId}/checkout	AUTH	CheckoutRequest{metodoPago}	Confirma la compra: transforma el carrito en un Pedido (ver flujo detallado en sección 8). Lanza CarritoVacioException / StockInsuficienteException

8. DetalleCarritoController

controllers/DetalleCarritoController.java · @RequestMapping("DetalleCarrito")

Endpoint	Seguridad	Body	Detalle
GET /DetalleCarrito?carritoId	AUTH	—	Ítems del carrito
GET /DetalleCarrito/{carritoId}/{productoId}	AUTH	—	Un ítem puntual o 404
POST /DetalleCarrito	AUTH	DetalleCarritoRequest{carritoId, productId, cantidad}	Agrega un producto al carrito con el precio (con descuento) congelado en ese momento. DetalleCarritoDuplicateException / StockInsuficienteException
PUT /DetalleCarrito/{carritoId}/{productoId}	AUTH	DetalleCarritoRequest	Cambia la cantidad (recalcula precio con descuento vigente)
DELETE /DetalleCarrito/{carritoId}/{productoId}	AUTH	—	Quita el ítem del carrito

9. PedidoController

controllers/PedidoController.java · @RequestMapping("Pedido")

Endpoint	Seguridad	Body	Detalle
GET /Pedido?usuarioId&page&size	AUTH	—	Lista paginada de pedidos, opcionalmente por usuario
GET /Pedido/{pedidoId}	AUTH	—	Detalle o 404
GET /Pedido/numero/{numeroPedido}	AUTH	—	Búsqueda por número público (ej. PED-12)
POST /Pedido	AUTH	PedidoRequest	Alta manual de pedido (uso administrativo; el flujo normal es vía checkout)
PUT /Pedido/{pedidoId}	AUTH	PedidoRequest	Actualiza estado/montos/método de pago

POST /Pedido/{pedidoId}/cancelar	AUTH	—	Cancela el pedido: repone el stock de cada producto y marca estado = CANCELADO (idempotente). Orquestado por PedidoOrchestratorServiceImpl
DELETE /Pedido/{pedidoId}	AUTH	—	Elimina el pedido, repone stock y borra sus DetallePedido

10. DetallePedidoController

controllers/DetallePedidoController.java · @RequestMapping("DetallePedido")

Endpoint	Seguridad	Body	Detalle
GET /DetallePedido?pedidoId	AUTH	—	Ítems de un pedido
GET /DetallePedido/{detallePedidoId}	AUTH	—	Detalle o 404
POST /DetallePedido	AUTH	DetallePedidoRequest DetallePedidoDuplicateException	Alta manual de línea de pedido.
PUT /DetallePedido/{detallePedidoId}	AUTH	DetallePedidoRequest	Actualiza cantidad/observaciones
DELETE /DetallePedido/{detallePedidoId}	AUTH	—	204 / 404

8. Flujos de negocio end-to-end

8.1 Registro, login y navegación del catálogo

- 1 Un visitante navega el catálogo **sin necesidad de loguearse**: GET /Producto (con filtros por categoría/nombre/precio y paginación), GET /Categories, GET /Imagen, GET /Descuento.
- 2 Para comprar, se registra con POST /Auth/register (o inicia sesión con POST /Auth/authenticate si ya tiene cuenta) y recibe un JWT.
- 3 A partir de ahí, incluye el token en el header Authorization de cada request protegida.

8.2 Armado del carrito

- 1 Crea un carrito con POST /Carrito {usuarioId, fechaCarrito}.
- 2 Agrega productos con POST /DetalleCarrito {carritoId, productoId, cantidad}. El service valida que no exista ya ese producto en el carrito (si existe, hay que usar PUT para cambiar cantidad) y que haya stock suficiente. El precio unitario se calcula **en ese momento** con DescuentoService.getPrecioConDescuento() y queda "congelado" en el DetalleCarrito — cambios posteriores en el descuento no afectan lo ya agregado.
- 3 Puede ajustar cantidades (PUT, revalida stock y recalcula precio) o quitar ítems (DELETE) en cualquier momento antes de pagar.

8.3 Checkout (el corazón del flujo de compra)

POST /Carrito/{carritoId}/checkout — todo ocurre dentro de una única transacción (@Transactional(rollbackFor = Throwable.class)) en CheckoutServiceImpl: si cualquier paso falla, se revierte todo.

- 1 Busca el carrito; si no existe → CarritoNotFoundException (404).
- 2 Trae sus ítems (DetalleCarrito); si está vacío → CarritoVacioException (400).
- 3 Revalida el stock de cada ítem contra el stock **actual** del producto (puede haber cambiado desde que se agregó al carrito); si falta → StockInsuficienteException (400).
- 4 Calcula subtotal = $\sum (\text{cantidad} \times \text{precioUnitario})$ de todos los ítems.
- 5 Crea el Pedido en estado PENDIENTE, con numeroPedido = "PED-" + id, usando el metodoPago recibido en el body.
- 6 Copia cada DetalleCarrito a un DetallePedido nuevo (misma cantidad y precio).
- 7 Descuenta stock real de cada Producto (ajustarStock(id, -cantidad)).
- 8 Vacía el carrito (borra sus DetalleCarrito) para dejarlo listo para una próxima compra.
- 9 Devuelve el Pedido creado (200 OK).

8.4 Seguimiento y cancelación del pedido

- 1 El usuario consulta sus pedidos con GET /Pedido?usuarioId=, por id (GET /Pedido/{id}) o por número público (GET /Pedido/numero/{numeroPedido}).
- 2 Si necesita cancelar, POST /Pedido/{id}/cancelar — PedidoOrchestratorServiceImpl recorre los DetallePedido y **repone el stock** de cada producto (ajustarStock(id, +cantidad)) antes de marcar el pedido como CANCELADO. Si ya estaba cancelado, la operación es idempotente (no vuelve a reponer stock).
- 3 DELETE /Pedido/{id} hace lo mismo (repone stock, borra los DetallePedido) pero elimina el registro del pedido por completo, en vez de solo marcarlo cancelado.

8.5 Administración (rol ADMIN)

- 1 Gestión de catálogo: alta/edición/baja de Category, Producto (incluye PATCH .../stock para reponer inventario) e Imagen (URL externa o subida de archivo binario vía /Imagen/upload).
- 2 Gestión de promociones: alta/edición/baja de Descuento por producto, con rango de fechas y validación de porcentaje 0-100.
- 3 Gestión de cuentas: listar todos los usuarios, crear usuarios manualmente, y **otorgar/revocar el rol ADMIN** con PATCH /Usuario/{id}/permisos?role=ADMIN.

9. Configuración del proyecto

application.properties

```
# Puerto del servidor embebido (Tomcat)
server.port=4002

# Conexión a MySQL local, base "marketplace"
spring.datasource.url=jdbc:mysql://localhost:3306/marketplace
spring.datasource.username=root
spring.datasource.password=***** // redactado en este documento

# Hibernate crea/actualiza el esquema automáticamente al arrancar
spring.jpa.hibernate.ddl-auto=update

# Clave HMAC para firmar los JWT + expiración (24hs en milisegundos)
application.security.jwt.secretKey=***** // redactado
application.security.jwt.expiration=86400000

# Límite de tamaño para subida de imágenes (Imagen/upload)
spring.servlet.multipart.max-file-size=5MB
spring.servlet.multipart.max-request-size=5MB
```

Observación de seguridad: la clave JWT y la contraseña de la base de datos están hardcodeadas en texto plano en `application.properties`, el cual está versionado en el repositorio. Para un entorno real conviene moverlas a variables de entorno o un secret manager, y excluir el archivo (o al menos esos valores) del control de versiones.

Dependencias principales (pom.xml)

Dependencia	Para qué se usa
spring-boot-starter-parent 4.1.0	BOM de versiones y configuración base de Spring Boot
spring-boot-starter-webmvc	Servidor web MVC / REST embebido (Tomcat)
spring-boot-starter-data-jpa	Spring Data JPA + Hibernate como implementación
mysql-connector-java 8.0.33	Driver JDBC para conectar con MySQL
spring-boot-starter-security	Autenticación/autorización, filtros, PasswordEncoder
jjwt-api / jjwt-impl / jjwt-jackson 0.12.5	Generación y validación de tokens JWT
spring-boot-starter-actuator	Endpoints de monitoreo/salud de la aplicación
lombok	Reduce boilerplate (getters/setters/constructores/builders)
spring-boot-devtools	Recarga automática en desarrollo

Cómo correr el proyecto

- 1 Tener MySQL corriendo localmente y una base llamada marketplace (o dejar que `ddl-auto=update` cree las tablas al arrancar, si la base ya existe).
- 2 Ajustar usuario/contraseña de MySQL en `application.properties` si difieren de los actuales.
- 3 Ejecutar con Maven: `./mvnw spring-boot:run` (o desde el IDE). La API queda disponible en `http://localhost:4002`.
- 4 Registrar un usuario, promoverlo a ADMIN manualmente en la base (o vía otro admin) para poder cargar catálogo.